Episodio 3 de 14: Los rivales: candados, cuentas dobles y basura retrasada

Serie «Leer sin que te quiten el libro»: la réplica del paper «A Fast Wait-Free Solution to Read-Reclaim Races in Reference Counting» (Euro-Par 2024), explicada para principiantes.

- **Episodio anterior:** episodio 2, «La carrera entre leer y liberar».
- **Episodio siguiente:** episodio 4, «La solución del paper: el taco de vales».

Lo esencial de este episodio:

- Mutex y Libstdc++ usan un candado: es simple, pero forma colas.
- Williams y Folly guardan una cuenta junto al puntero, pero usan bucles de CAS que pueden fallar y reintentar.
- Anderson retrasa la liberación con hazard pointers: es rápido, pero retiene algo de memoria un rato.

Parte 3. Los algoritmos, uno por uno

Vamos de lo más simple a lo más sofisticado. Todos resuelven el mismo problema de la Parte 1, y para cada uno verás: la idea en una frase, la analogía, el paso a paso, y qué lo hace bueno o malo.

3.0 La versión ingenua (la que está rota)

Idea: leer el puntero y luego sumar 1 al contador.

Es exactamente lo de la Parte 1. Está en `codigo/estres_asan.cpp` solo para demostrar el fallo: AddressSanitizer, un detector de errores de memoria, lo atrapa en cuanto se ejecuta (ver Parte 6.5). No aparece en el paper como rival porque no es una solución, es el problema.

3.1 Mutex (referencia del equipo, no está en el paper)

Idea en una frase: una sala con una sola llave: solo entra uno.

Analogía: el tablón está en una sala cerrada. Para mirarlo o cambiarlo pides la llave. Si está ocupada, **te sientas a dormir** y te despiertan cuando quede libre.

`load`, paso a paso:

1. Pedir la llave. Si no está libre, el sistema operativo te duerme.
2. Leer el puntero.
3. Sumar 1 al contador. Nadie puede cambiar nada: estás solo.
4. Devolver la llave.

`store`, paso a paso:

1. Pedir la llave.
2. Cambiar el puntero.
3. Devolver la llave.

4. Soltar el objeto viejo, ya fuera de la sala, para no retener a nadie.

Garantía	Bloqueante
Memoria extra	Ninguna; libera al instante
A favor	Muy simple y correcta. Dormir evita gastar CPU
En contra	Se forman colas; si quien tiene la llave se para, todos esperan. Dormir y despertar es lento, por eso su p99 es malísimo

3.2 Libstdc++ (`std::atomic<std::shared_ptr>` de GCC)

Idea en una frase: un pestillo diminuto dentro del propio puntero, y quien no entra se queda dando vueltas a la puerta.

Analogía: es como el Mutex, pero el «cerrado» es un puntito en la esquina del propio tablón. No hay sala de espera: quien quiere entrar **se queda de pie mirando** el puntito hasta que se apaga.

El truco técnico. Los objetos están alineados en memoria, así que el último bit de un puntero siempre vale 0. GCC aprovecha ese bit como pestillo: `0` libre, `1` ocupado.

`load`, paso a paso (`bits/shared_ptr_atomic.h`):

1. Mirar el bit. Si está a 1, esperar dando vueltas.
2. Con un `CAS`, ponerlo a 1. Si otro se adelantó, volver al paso 1.
3. Leer el puntero y sumar 1 al contador.
4. Restar 1 a la palabra para apagar el bit.

`store`, paso a paso:

1. Encender el bit igual que antes.
2. Con un `exchange`, poner el puntero nuevo; eso además apaga el bit.
3. Soltar el objeto viejo.

Garantía	Bloqueante
Memoria extra	Ninguna; libera al instante
A favor	Viene con el compilador, y sin contención es muy ágil
En contra	Cada lectura escribe dos veces en la línea más disputada. Si quien tiene el bit pierde su turno de CPU, todos dan vueltas en vano

Con un solo hilo es rápido, porque nadie compite. En cuanto hay contención se hunde, y por eso en el paper pasa de 7 Mop/s con un hilo a 0,1 con 144.

3.3 Williams (split reference count con 16 bytes)

Idea en una frase: apuntarse en el tablón *antes* de ir a la libreta, para que nadie tire el libro mientras vas de camino.

Analogía: el tablón tiene dos columnas: la ficha del libro y una rayita por cada persona que **va de camino**. Antes de ir, haces una rayita. Cuando ya sumaste 1 en la libreta, borras tu rayita. Si Beto cambia el tablón, ve las rayitas y sabe cuánta gente viene en camino, y las pasa a la libreta para que el libro no se destruya.

A esto se le llama **contador partido** (*split reference count*): una parte de la cuenta vive junto al puntero (la **local**) y otra en la libreta (la **global**).

Lo que hay físicamente: puntero de 8 bytes más 8 bytes de contadores, en total **16 bytes**, que se manejan con `DWCAS`, la instrucción `cmpxchg16b`.

`load`, paso a paso (`local_access` en `codigo/williams/atomic_shared_ptr`):

1. Leer los 16 bytes.
2. Bucle `DWCAS`: sumar 1 a `access_count`. Si alguien tocó el tablón, repetir.
3. Sumar 1 en la libreta (ya no hay peligro: mi rayita protege el objeto).
4. Bucle `DWCAS`: restar 1 a `access_count`.
5. Si el puntero cambió mientras tanto, avisar en la libreta con `remove_external_counter`.

`store`, paso a paso:

1. Intercambiar los 16 bytes por los nuevos.
2. Pasar las rayitas que quedaban a la libreta del objeto viejo.
3. Restar la referencia que tenía el tablón.

Garantía	Lock-free
Memoria extra	Ninguna; libera al instante
A favor	Sin candados: nadie bloquea a nadie
En contra	Dos bucles de `DWCAS` por lectura, además de la libreta. Mucho trabajo por operación

Fíjate en lo importante: **cada lectura toca el tablón dos veces**. Los Autores lo tocan una sola.

Una nota práctica. Esos 16 bytes solo se pueden cambiar de golpe con la instrucción `cmpxchg16b`, y el compilador solo la usa si se le pide con `-mcx16`. Sin esa bandera, `boost::atomic` mete un candado por dentro, sin avisar, y Williams deja de ser lock-free. Nos pasó: ver la Parte 6.2.

3.4 Folly (Facebook/Meta)

Idea en una frase: Williams, pero metiéndolo todo en 8 bytes y repartiendo las rayitas en lote.

Analogía: en lugar de un tablón doble, Folly aprovecha que los punteros reales solo usan 48 de sus 64 bits. En los 16 bits libres cabe el contador local. Además, el libro ya tiene anotada de antemano una **provisión de 8 192 usuarios** en su libreta, así que las lecturas casi nunca necesitan ir a ella.

`load`, paso a paso (`takeOwnedBase` en `AtomicSharedPtr.h`):

1. Leer los 8 bytes.

2. Bucle `CAS` para sumar 1 al contador local. Si otro se adelantó, repetir.
3. Si el contador local pasa de 4 096, trasladar un lote de 4 096 a la libreta y restarlo del contador local.
4. Si llegara a 8 192 (más lectores simultáneos que provisión), **esperar** cediendo el turno.

`store`, paso a paso:

1. Dar al objeto nuevo su provisión de 8 192 en la libreta.
2. Con un `exchange`, poner el nuevo y quedarse con el viejo.
3. Ajustar la libreta del viejo según cuántas lecturas se consumieron.

Garantía	Lock-free; bloqueante en el caso extremo de más de 8 192 hilos
Memoria extra	Ninguna; libera al instante
A favor	Una sola operación de 8 bytes por lectura, y la libreta casi no se toca
En contra	Usa `CAS` en bucle: **puede perder y reintentar**. Con mucha contención, ese bucle domina

Es casi la misma idea que los Autores. La gran diferencia es una sola instrucción: Folly usa `CAS` (puede fallar), los Autores usan `fetch_add` (no falla). Folly hace esto porque necesita comprobar que no se desborda su contador de 16 bits.

Un aviso antes de ver los resultados. En el paper, Folly sale muy mal parado: con contención rinde entre 0,05× y 0,07× de lo que rinden los Autores. En esta réplica eso no pasa. No pasa con Docker, no pasa sin Docker y no pasa con la receta exacta del paper. La Parte 6.1 lo cuenta, y explica por qué la causa sigue abierta.

3.5 Anderson (hazard pointers)

Idea en una frase: antes de ir al libro, pon un cartel en tu mesa diciendo cuál usas; y quien quiera destruir algo, antes revisa todas las mesas.

Analogía: cada persona tiene una mesa con un cartelito. Quien va a usar el libro 42 escribe «42» en su cartelito. Cuando un bibliotecario va a tirar algo, no lo tira: lo mete en una **caja de pendientes**. Cada 2 000 cosas en la caja, recorre todas las mesas, y solo destruye lo que no aparezca en ningún cartelito.

Un **hazard pointer** es ese cartelito: un puntero que declara «ojo, estoy usando esto».

Un detalle muy fino, que comprobamos en el código y en la prueba. El **objeto** se destruye al instante; lo que espera en la caja es solo el **bloque de control** (la libreta). La libreta debe sobrevivir porque un lector despistado podría aún intentar sumarle 1, y necesita una respuesta segura: «este libro ya no existe». Por eso esa suma usa un contador especial que, una vez en 0, ya no sube (`increment_strong_count_if_nonzero`).

`load`, paso a paso (`atomic_shared_ptr.hpp`):

1. Leer el puntero del tablón.
2. Escribirlo en mi cartelito (mi hazard pointer).
3. Volver a leer el tablón. Si cambió, volver al paso 2 con el valor nuevo.

4. Intentar sumar 1 al contador. Si estaba en 0, el objeto ya murió: repetir desde el paso 1.
5. Borrar mi cartelito.

`store`, paso a paso:

1. Con un `exchange`, poner el puntero nuevo.
2. Restar 1 al contador del viejo.
3. Si llega a 0: destruir el objeto y meter su bloque de control en mi caja.
4. Cada 2 000 retiradas, revisar los cartelitos de todos y liberar lo que nadie protege.

Garantía	Lock-free
Memoria extra	**Sí**: bloques de control pendientes, hasta unos 2 000 por hilo escritor o más
A favor	**Leer no escribe en el tablón**: varios lectores no se estorban ahí, y un `CAS` no falla porque otro haya leído
En contra	Memoria retenida. Cada `cleanup` recorre todos los hilos, y causa picos de latencia (medidos en la Parte 6.2)

Este punto explica mucho: en Autores, Folly y Williams **leer modifica el tablón**. En Anderson no. Por eso, cuando hay muchas actualizaciones sobre una sola pila, Anderson sale ganando: su `CAS` no falla por culpa de los lectores.

3.7 Los seis, de un vistazo

	Atómicas por lectura en la zona disputada	**¿Puede reintentar leyendo?**	**Garantía del `load`**	**Liberación**	**¿Leer escribe en el tablón?**
Mutex	candado + desbloqueo	espera dormido	Bloqueante	Inmediata	Sí (el candado)
Libstdc++	`CAS` + `fetch_sub`	sí, dando vueltas	Bloqueante	Inmediata	Sí
Williams	2 bucles de `DWCAS`	sí	Lock-free	Inmediata	Sí
Folly	1 bucle de `CAS`	sí	Lock-free	Inmediata	Sí
Anderson	ninguna en el tablón (solo lee)	sí, pocas veces	Lock-free	**Diferida** (bloque de control)	**No**
Autores	**1 `fetch_sub`**	**no** (salvo recarga, ≤ P)	**Wait-free**	**Inmediata**	Sí

Todos, además, deben sumar y restar en la libreta del objeto cuando hace falta. Y aquí viene una diferencia importante en Autores: al **terminar** de usar el objeto se resta 1 en la libreta. Así que, con una única pila y muchos lectores, hay **dos** líneas calientes: el tablón y la libreta. En Anderson solo hay una, la libreta. Es una explicación razonable de por qué Anderson acaba ganando sobre una sola pila, aunque ojo: con u = 1 %, en el paper eso solo ocurre a partir de unos 54 hilos, y en esta máquina no llega a pasar (ver Parte 6.1).
